

Y16 입력이 화면에 표시되기까지

카메라가 16비트 그레이스케일(Y16)로 출력할 때, 프레임을 받아서 QLabel 에 그리기까지 거치는 연산을 코드 위치 순서대로 정리했습니다. 줄 번호는 현재 작업 트리 기준입니다.

0x0000		Y16		0xFFFF	
입력 포맷		요청 해상도		표시 포맷	
Y16 · CV_16UC1		640 × 480		Grayscale8	
				변환 방식	
				min-max 선형	

한눈에 보기

캡처 스레드

openDevice → VideoCapture::read → QImage 복사
→ emit frameReady

Qt queued connection — 스레드 경계를 넘어 GUI 이벤트 루프로 전달

GUI 스레드

onFrameReady → normalize16To8 → scaled + setPixmap
→ updateFps

단계별 상세

1

카메라 장치 열기

GUI 스레드

파일	줄	내용
Viewer.cpp	230-290	on_btnCamConnect_clicked() — Connect 버튼 핸들러
Viewer.cpp	257	Windows: openDevice(camIndex, 640, 480)
Viewer.cpp	262-264	Linux: openDefaultCamera("CLTC_T_VG", 640, 480, Y16)
Capture.cpp	12-25	Windows openDevice (DirectShow)
Capture.cpp	28-44	Linux openDevice (V4L2)
Capture.cpp	127-141	queryFormat() — 실제 적용된 폭·높이·FOURCC 조회

파일	줄	내용
Viewer.cpp	288	m_capThread->startCapture() — 캡처 루프 시작

Capture.cpp · Windows

```

14  if (!m_cap.open(index, cv::CAP_DSHOW)) { ... }
19  m_cap.set(cv::CAP_PROP_FRAME_WIDTH, w);
20  m_cap.set(cv::CAP_PROP_FRAME_HEIGHT, h);
21  m_cap.set(cv::CAP_PROP_CONVERT_RGB, 0);
22  m_cap.set(cv::CAP_PROP_FOURCC, cv::VideoWriter::fourcc('Y','1','6',' '));

```

- 백엔드는 Windows에서 CAP_DSHOW, Linux에서 CAP_V4L2 입니다.
- CAP_PROP_CONVERT_RGB = 0 으로 OpenCV의 BGR 자동 변환을 끕니다. 그래서 16비트 raw 값이 CV_16UC1 Mat으로 그대로 들어옵니다.
- Linux에서는 /dev/video* 중 카드 이름에 CLTC_T_VG 가 들어간 장치를 자동으로 고릅니다 (Capture.cpp 62-80).

2

프레임 읽기

캡처 스레드

파일	줄	내용
Capture.cpp	89-97	getFrame() — m_cap.read(frame) 로 한 프레임 획득
Capture.cpp	98	frame.type() == CV_16UC1 이면 Y16 분기로 진입

Capture.cpp

```

92  cv::Mat frame;
93  if (!m_cap.read(frame)) {
94      qWarning() << "⚠ Failed to grab frame";
95      return QImage();
96  }
98  if (frame.type() == CV_16UC1) {

```

- 픽셀 값은 바뀌지 않습니다. 16비트 단일 채널 버퍼를 그대로 받습니다.
- 읽기에 실패하면 null QImage 를 반환하고, 캡처 스레드는 그 프레임을 건너뛵니다.

QImage(Grayscale16)로 감싸고 복사 캡처 스레드

파일	줄	내용
Capture.cpp	99-101	Mat 버퍼를 Format_Grayscale16 QImage로 감싼 뒤 .copy()

```
Capture.cpp
99 // === Y16 ===
100 return QImage((const uchar*)frame.data, frame.cols, frame.rows,
101               frame.step, QImage::Format_Grayscale16).copy();
```

- frame.step (한 줄의 바이트 수)을 bytesPerLine 으로 넘겨서 Mat 메모리를 그대로 가리키게 합니다.
- frame 은 함수가 끝나면 해제되므로 .copy() 로 깊은 복사를 합니다(640×480×2 = 614,400 바이트).

캡처 루프와 시그널 전달 캡처 스레드

파일	줄	내용
Capturethread.h	35-43	run() — m_running 이 true인 동안 반복
Viewer.cpp	41-42	frameReady → Viewer::onFrameReady 연결

```
Capturethread.h
35 void run() override {
36     while (m_running) {
37         QImage frame = m_cap->getFrame();
38         if (!frame.isNull()) {
39             emit frameReady(frame.copy());
40         }
41         msleep(1);
42     }
43 }
```

- emit할 때 frame.copy() 로 한 번 더 깊은 복사를 합니다.
- 송신 쪽(QThread)과 수신 쪽(Viewer)의 스레드가 다르므로 기본값인 AutoConnection이 **queued**로 동작합니다. 그래서 onFrameReady 는 GUI 스레드의 이벤트 루프에서 실행됩니다.
- 반복마다 msleep(1) 로 1ms를 씁니다. 실제 프레임 속도는 read() 가 블로킹되는 시간이 정합니다.

프레임 수신과 포맷 분기 GUI 스레드

파일	줄	내용
Viewer.cpp	293-324	onFrameReady(const QImage&)

파일	줄	내용
Viewer.cpp	295-296	Format_Grayscale16 이면 normalize16To8(frame, true) 호출

Viewer.cpp

```

293 void Viewer::onFrameReady(const QImage& frame) {
294     QImage displayImg;
295     if (frame.format() == QImage::Format_Grayscale16) {
296         displayImg = normalize16To8(frame, true);
                // ...
302     } else {
303         displayImg = frame;
304     }

```

- Y16이 아닌 포맷(UYVY→RGB888, BGR888)은 getFrame() 에서 이미 RGB로 바꿔 두었기 때문에 그대로 표시합니다.

6

16비트 → 8비트 정규화

GUI 스레드

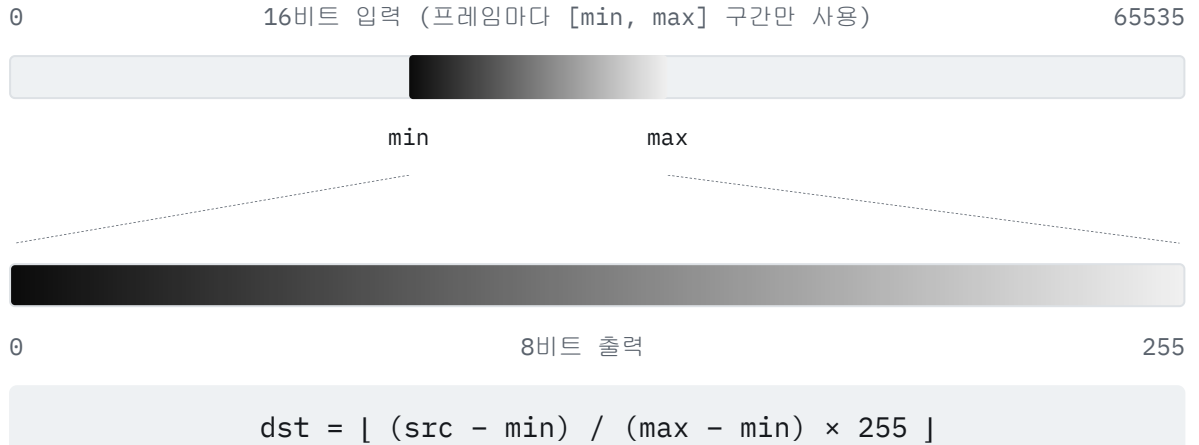
파일	줄	내용
Viewer.h	519	normalize16To8 선언 (자유 함수)
Viewer.cpp	330-353	normalize16To8(const QImage&, bool autoStretch) 구현
Viewer.cpp	336-345	1차 순회: 프레임 전체의 min/max 계산
Viewer.cpp	346-351	2차 순회: 픽셀마다 선형 매핑 후 Grayscale8 에 기록

Viewer.cpp

```

335 const uint16_t *src = reinterpret_cast<const uint16_t*>(img16.constBits());
336 uint16_t minVal = autoStretch ? 65535 : 0;
337 uint16_t maxVal = autoStretch ? 0 : 16383;
338 if (autoStretch) {
339     for (int i = 0; i < pixels; i++) {
340         uint16_t v = src[i];
341         if (v < minVal) minVal = v;
342         if (v > maxVal) maxVal = v;
343     }
344     if (maxVal == minVal) maxVal++;
345 }
346 QImage img8(width, height, QImage::Format_Grayscale8);
347 uchar *dst = img8.bits();
348 for (int i = 0; i < pixels; i++) {
349     float norm = float(src[i] - minVal) / float(maxVal - minVal);
350     dst[i] = static_cast<uchar>(norm * 255.0f);
351 }

```



- **1차 순회**에서 프레임마다 min/max를 새로 구합니다. 프레임 단위 자동 대비 조정(AGC)과 같은 효과입니다. 모든 픽셀 값이 같으면 0으로 나누지 않도록 `maxVal++` 를 합니다.
- **2차 순회**에서 `float` 로 정규화하고 255를 곱한 뒤 `static_cast<uchar>` 로 소수점을 버립니다. 그래서 255가 나오는 건 `src == max` 인 픽셀뿐입니다.
- 히스토그램 평활화, 감마, LUT, 컬러맵 같은 비선형 처리는 없습니다. 순수한 min-max 선형 스트레치입니다.
- `autoStretch=false` 이면 0-16383 고정 범위(14비트)를 쓰도록 되어 있지만, 지금은 이렇게 호출하는 곳이 없습니다.

스케일링과 화면 출력

GUI 스레드

파일	줄	내용
Viewer.cpp	307-312	메인 <code>videoLabel</code> 에 출력 (팝업이 닫혀 있을 때만)
Viewer.cpp	315-320	비디오 팝업 <code>m_popupLabel</code> 에 출력
Viewer.cpp	76-93	<code>setupVideoLabel()</code> — 클릭 가능한 라벨로 교체, 고정 크기
Viewer.cpp	370-403	<code>onVideoLabelClicked()</code> — 팝업 토글, <code>m_videoPopupOnlyUpdate</code> 설정

Viewer.cpp

```

307 if (!m_videoPopupOnlyUpdate) { // false일 때만 videoLabel 업데이트
308     ui->videoLabel->setPixmap(QPixmap::fromImage(displayImg).scaled(
309         ui->videoLabel->size(),
310         Qt::KeepAspectRatio,
311         Qt::SmoothTransformation));
312 }
315 if (m_popupLabel) {
316     m_popupLabel->setPixmap(QPixmap::fromImage(displayImg).scaled(
317         m_popupLabel->size(),
318         Qt::KeepAspectRatio,
319         Qt::SmoothTransformation));
320 }
```

- `QPixmap::fromImage` 로 표시용 픽스맵을 만들고, `SmoothTransformation (bilinear)`으로 라벨 크기에 맞춰 비율을 유지하며 리샘플링합니다.
- 팝업이 열려 있으면 `m_videoPopupOnlyUpdate = true` (402줄)가 되어 메인 라벨은 갱신하지 않습니다.
- 팝업 라벨은 `setScaledContents(true)` (386줄)도 켜져 있어서, `scaled()` 결과를 `QLabel`이 한 번 더 늘리거나 줄입니다.

8

FPS 갱신

GUI 스레드

파일	줄	내용
Viewer.cpp	323	<code>if (m_showFps) updateFps();</code>
Viewer.cpp	355-364	<code>updateFps()</code> - 1초마다 <code>frameCount / 경과초</code> 를 <code>fpsEntry</code> 에 표시
Viewer.cpp	284-286	Connect할 때 카운터와 타이머 초기화

- FPS는 **GUI에서 처리를 끝낸 프레임 수** 기준입니다. GUI 스레드가 밀리면 카메라의 실제 출력 속도보다 낮게 보일 수 있습니다.

검토할 점

GUI 스레드 부하

Viewer.cpp 293-324

정규화의 픽셀 전체 순회 2회와 Smooth 스케일링이 모두 GUI 스레드에서 실행됩니다. 프레임이 쌓이면 UI 반응이 느려질 수 있으니, 필요하면 캡처 스레드에서 8비트 변환까지 끝내고 넘기는 방식을 고려하세요.

중복 깊은 복사

Capturethread.h 39

`getFrame()` 이 이미 `.copy()` 로 반환하므로 `emit frameReady(frame.copy())` 의 복사는 필요 없습니다. QImage는 암시적 공유라서 `queued signal`로 넘길 때 따로 복사할 필요가 없습니다.

줄 끝 패딩 미고려

Viewer.cpp 339, 348

`width × height` 로 연속 인덱싱을 하고 `bytesPerLine()` 을 보지 않습니다. 640폭에서는 32비트 정렬 패딩이 없어서 문제가 없지만, 폭이 4의 배수가 아니면 영상이 비스듬하게 틀어질 수 있습니다.

상위 비트 마스킹 없음

Viewer.cpp 340, 349

고정 범위 모드는 14비트(0-16383)를 전체로 하지만, auto-stretch 모드는 16비트 값 전체로 min/max를 구합니다. 센서가 상위 2비트에 다른 정보를 넣는다면 스트레치 범위가 틀어질 수 있습니다.